

```

1  /* =====
2  APUNTES – GUIA RAPIDA PROGRAMACION 3 (PUCP)
3  Un solo archivo compilable. Ojo: en un .java solo UNA clase o
4  interfaz puede ser 'public', y debe llamarse igual que el archivo.
5  Por eso aqui solo 'Apuntes' es public; el resto va sin modificador.
6  ===== */
7
8  import java.util.Date;
9  import java.text.SimpleDateFormat;
10 import java.time.LocalDate;
11 import java.time.LocalDateTime;
12 import java.time.format.DateTimeFormatter;
13 import java.util.Random;
14 import java.util.List;
15 import java.util.ArrayList;
16
17 public class Apuntes {
18
19     // 'throws Exception' es OBLIGATORIO si vas a usar sdf.parse().
20     // Los solucionarios de lab lo llevan por eso mismo.
21     public static void main(String[] args) throws Exception {
22
23         // =====
24         // 1. FECHAS Y HORAS
25         // =====
26
27         // --- Date (legacy, es la que usan los labs de la PUCP) ---
28         Date fechaActual = new Date();
29
30         SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy HH:mm");
31
32         // FORMAT: Date -> String (para imprimir)
33         System.out.println("Formateada: " + sdf.format(fechaActual));
34
35         // PARSE: String -> Date (para crear fechas a mano)
36         // Lanza ParseException, por eso el 'throws Exception' del main.
37         SimpleDateFormat sdfCorto = new SimpleDateFormat("dd-MM-yyyy");
38         Date fechaNacimiento = sdfCorto.parse("02-11-1992");
39         System.out.println("Parseada: " + sdfCorto.format(fechaNacimiento));
40
41         // --- java.time (Java 8+) ---
42         LocalDate soloFecha = LocalDate.now();
43         LocalTime soloHora = LocalTime.now();
44         LocalDateTime fechaYHora = LocalDateTime.now();
45
46         // Hora fija: LocalTime.of(hora, minuto, segundo)
47         LocalTime inicioClase = LocalTime.of(8, 0, 0);
48
49         DateTimeFormatter dtf = DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss");
50         System.out.println("Moderna: " + fechaYHora.format(dtf));
51         System.out.println("Clase a las " + inicioClase + " del " + soloFecha
52             + " (hora actual " + soloHora.getHour() + "h)");
53
54         // =====
55         // 2. RANDOM
56         // =====
57
58         // Se instancia UNA vez, fuera de cualquier bucle.
59         Random rand = new Random();
60
61         int n = 10;
62         int aleatorio = rand.nextInt(n); // entero de 0 a 9 (el tope NO se incluye)
63         System.out.println("Aleatorio 0-9: " + aleatorio);
64         // TRAMPA: nextInt(0) lanza IllegalArgumentException.
65
66         // =====
67         // 3. LISTAS: agregar y eliminar
68         // =====
69
70         List<String> frutas = new ArrayList<>();
71         frutas.add("Manzana");
72         frutas.add("Platano");
73         frutas.add("Uva");
74
75         frutas.remove(1); // por INDICE -> saca "Platano"
76         frutas.remove("Uva"); // por OBJETO -> busca y saca "Uva"
77         System.out.println("Frutas: " + frutas);
78
79         // TRAMPA CLASICA: con List<Integer> los dos remove se confunden.
80
81
82

```

```

83 List<Integer> numeros = new ArrayList<>(List.of(10, 20, 30));
84 numeros.remove(1); // INDICE 1 -> saca el 20
85 numeros.remove(Integer.valueOf(30)); // OBJETO 30 -> saca el 30
86 System.out.println("Numeros: " + numeros);
87
88 // TRAMPA 2: remove(Object) compara con equals(). Si tu clase no
89 // sobrescribe equals(), solo elimina si es LA MISMA referencia.
90
91 // TRAMPA 3: List.of(...) crea una lista INMUTABLE.
92 List<String> fija = List.of("a", "b");
93 try {
94     fija.add("c");
95 } catch (UnsupportedOperationException e) {
96     System.out.println("List.of es immutable: no admite add/remove");
97 }
98 // Solucion: envolverla -> new ArrayList<>(List.of("a","b"))
99
100
101 // =====
102 // 4. ENUM
103 // =====
104 // Conjunto cerrado de valores con nombre. Se usa cuando el
105 // atributo solo puede tomar ciertas opciones fijas.
106
107 Modalidad m = Modalidad.PRESENCIAL;
108 System.out.println("Modalidad: " + m); // imprime PRESENCIAL
109 System.out.println("Es virtual? " + (m == Modalidad.VIRTUAL));
110 // Comparar enums con == es correcto (no hace falta equals).
111
112
113 // =====
114 // 5. POLIMORFISMO: lista del tipo del padre / de la interfaz
115 // =====
116 // Guardas objetos de clases distintas en la misma lista, siempre
117 // que compartan la superclase o la interfaz.
118
119 List<Miembro> miembros = new ArrayList<>();
120 miembros.add(new Alumno("Juan Perez", fechaNacimiento, "20090606", 68.3));
121 miembros.add(new Profesor("Andrea Montenegro", new Date(), "64891", Estado.TC));
122 miembros.add(new MiembroExterno("Jorge Gonzales", new Date(), Modalidad.VIRTUAL));
123
124 // Cada objeto responde con SU version del metodo. Eso es polimorfismo.
125 for (Miembro miembro : miembros) {
126     System.out.print(miembro.presentarse());
127 }
128
129
130 // =====
131 // 6. instanceof + DOWNCAST (el corazon de los labs)
132 // =====
133 // La lista es de Miembro, pero solo los MiembroPUCP tienen
134 // consultarDatos(). Se filtra con instanceof y se castea.
135
136 System.out.println("--- Solo miembros PUCP ---");
137 for (Miembro miembro : miembros) {
138     if (miembro instanceof MiembroPUCP) {
139         // Downcast en linea, que es como lo escriben los solucionarios.
140         // Los parentesis de AFUERA son obligatorios: primero se castea
141         // el objeto y recién sobre el resultado se llama al metodo.
142         System.out.print(((MiembroPUCP) miembro).consultarDatos());
143     }
144 }
145 // Sin esos parentesis externos:
146 // (MiembroPUCP) miembro.consultarDatos()
147 // Java entiende "llama consultarDatos() y castea lo que devuelva",
148 // e intenta invocar el metodo sobre un Miembro -> ni siquiera compila.
149 //
150 // Equivalente en dos pasos, por si te resulta mas legible:
151 // MiembroPUCP p = (MiembroPUCP) miembro;
152 // System.out.print(p.consultarDatos());
153 //
154 // TRAMPA: castear sin preguntar con instanceof lanza ClassCastException.
155
156
157 // =====
158 // 7. STATIC (el correlativo automatico)
159 // =====
160 // El contador vive en la CLASE, no en cada objeto: es uno solo
161 // compartido por todas las instancias. Ver Miembro mas abajo.
162
163 System.out.println("Miembros creados hasta ahora: " + Miembro.getTotalCreados());
164 }

```

```

165 }
166
167
168 /* =====
169 ENUMS
170 ===== */
171
172 enum Modalidad {
173     PRESENCIAL, VIRTUAL
174 }
175
176 enum Estado {
177     TPA, // tiempo parcial por asignatura
178     TC, // tiempo completo
179     TPC // tiempo parcial por convenio
180 }
181
182
183 /* =====
184 INTERFAZ
185 -----
186 Es un CONTRATO: dice que metodos hay que tener, no como se hacen.
187 Sus metodos son public abstract de forma implicita.
188
189 OJO CON EL VOCABULARIO: las clases que ponen 'implements' NO son
190 "clases hijas" de la interfaz – la IMPLEMENTAN. Herencia (extends)
191 es de UNA sola clase; interfaces puedes implementar VARIAS.
192 ===== */
193
194 interface IConsultable {
195     String consultarDatos();
196 }
197
198
199 /* =====
200 CLASE ABSTRACTA vs INTERFAZ (pregunta segura de examen)
201 -----
202 CLASE ABSTRACTA | INTERFAZ
203 -----|-----
204 No se puede instanciar | No se puede instanciar
205 Puede tener atributos normales | Solo constantes (static final)
206 Puede tener constructor | No tiene constructor
207 Mezcla metodos hechos y abstractos | Solo firmas (salvo default)
208 Se hereda con 'extends' | Se adopta con 'implements'
209 Solo UNA por clase | VARIAS por clase
210 -----|-----
211 Regla practica: clase abstracta cuando comparten ESTADO y codigo;
212 interfaz cuando solo comparten una CAPACIDAD.
213 ===== */
214
215 abstract class Miembro {
216
217     // STATIC: existe una sola copia para toda la clase.
218     private static int correlativo = 1;
219
220     private int codigo;
221     private String nombre;
222     private Date fechaNacimiento;
223
224     public Miembro(String nombre, Date fechaNacimiento) {
225         this.codigo = correlativo++; // asigna y luego incrementa el contador
226         this.nombre = nombre;
227         this.fechaNacimiento = fechaNacimiento;
228     }
229
230     // Metodo abstracto: obliga a cada hija a dar su propia version.
231     public abstract String presentarse();
232
233     // Metodo concreto: lo heredan todas tal cual.
234     public String getNombre() {
235         return nombre;
236     }
237
238     public int getCodigo() {
239         return codigo;
240     }
241
242     public Date getFechaNacimiento() {
243         return fechaNacimiento;
244     }
245
246     // Metodo static: se llama con Miembro.getTotalCreados(), sin objeto.

```

```

247     public static int getTotalCreados() {
248         return correlativo - 1;
249     }
250 }
251
252
253 /* =====
254 NIVEL INTERMEDIO: abstracta que hereda Y ademas implementa interfaz
255 ===== */
256
257 abstract class MiembroPUCP extends Miembro implements IConsultable {
258
259     private String codigoPUCP;
260
261     public MiembroPUCP(String nombre, Date fechaNacimiento, String codigoPUCP) {
262         super(nombre, fechaNacimiento); // super() SIEMPRE va en la 1ra linea
263         this.codigoPUCP = codigoPUCP;
264     }
265
266     public String getCodigoPUCP() {
267         return codigoPUCP;
268     }
269
270     // Se deja abstracto de nuevo: lo resuelven Alumno y Profesor.
271     public abstract String consultarDatos();
272 }
273
274
275 /* =====
276 HOJAS: aquí se cierra la cadena de super() (3 niveles)
277 ===== */
278
279 class Alumno extends MiembroPUCP {
280
281     private double craest;
282
283     public Alumno(String nombre, Date fechaNacimiento, String codigoPUCP, double craest) {
284         super(nombre, fechaNacimiento, codigoPUCP); // llama a MiembroPUCP,
285         this.craest = craest; // que a su vez llama a Miembro
286     }
287
288     @Override
289     public String consultarDatos() {
290         return "Alumno: " + getCodigoPUCP() + " - " + getNombre() + " - CRAEST " + craest + "\n";
291     }
292
293     @Override
294     public String presentarse() {
295         return "[" + getCodigo() + "] Soy alumno: " + getNombre() + "\n";
296     }
297 }
298
299 class Profesor extends MiembroPUCP {
300
301     private Estado estado;
302
303     public Profesor(String nombre, Date fechaNacimiento, String codigoPUCP, Estado estado) {
304         super(nombre, fechaNacimiento, codigoPUCP);
305         this.estado = estado;
306     }
307
308     @Override
309     public String consultarDatos() {
310         return "Profesor: " + getCodigoPUCP() + " - " + getNombre() + " - " + estado + "\n";
311     }
312
313     @Override
314     public String presentarse() {
315         return "[" + getCodigo() + "] Soy profesor: " + getNombre() + "\n";
316     }
317 }
318
319 /* Esta NO implementa IConsultable: por eso el instanceof del main la
320 deja fuera del reporte. Ese es justamente el caso que hay que filtrar. */
321 class MiembroExterno extends Miembro {
322
323     private Modalidad dedicacion;
324
325     public MiembroExterno(String nombre, Date fechaNacimiento, Modalidad dedicacion) {
326         super(nombre, fechaNacimiento);
327         this.dedicacion = dedicacion;
328     }

```

```
329
330  @Override
331  public String presentarse() {
332      return "[" + getCodigo() + "] Soy externo: " + getNombre() + " (" + dedicacion + ")\n";
333  }
334 }
```